Scaled Engine

Overview of the tool, the delivery process, and what a shop can skip

Scaled Engine is a local-first project tool for a shop that runs more than one team on the same product. The name is a working title. The process is a light form of scaled agile: shared outcomes, a short planning cycle, team delivery, and a review of evidence. Agent work uses that same process, with extra gates where a feature needs data or ships an agent.

A person can hold more than one role. The steps do not merge. Before a task is saved, the tool asks which role the person is acting in.

Features

Organization

- Portfolio, value stream, delivery group, and team.
- Roles: stream owner, group facilitator, product manager, product owner, scrum master, technical lead, business stakeholder, and team member.
- One person can hold several roles. Each saved task stores the acting role.
- The delivery group can be a real group or a placeholder. The facilitator seat can be empty.

Work

- Goals, initiatives, features, and stories. An initiative moves through Proposed, Shaping, Ready, Active, and Done.
- Work kinds: feature, platform item, prototype, and experiment.
- A delivery path on every item: Discover, Define, Build, Check, Ship, with a history of moves.
- A shared done list. Check cannot finish while a required item on that list is open.
- A team board with a work-in-progress limit. A new start is refused when the board is full.

Planning and learning

- A planning cycle, six weeks unless the group chooses other dates.
- Group outcomes and team outcomes, milestones, dependencies, risks, and a confidence check.
- Each dependency has an owner, a needed-by date, and an open date. The providing team is optional.
- Dependent work only when a feature needs another team: what must change, what done looks like, and a needed-by date. The shop may name the team or leave it blank.
- An architecture review, and a PII review, only on features that say the review applies.
- Iteration board, team sync, cross-team note, and a retrospective that stays on the team.
- An evidence review at each iteration boundary: what was observed, then what will change.

Measures and controls

- A delay-cost score for sorting the backlog, from value, urgency, risk or opportunity, and size.

- Story sizing by team: t-shirt sizes or points. T-shirt labels start at 1, 2, 3, 5, and 8, and the team can change those numbers.
- Team velocity: the estimate completed in an iteration, with the last three iterations visible.
- Throughput, lead time, work in progress, active time versus wait, work mix, and outcome reliability.
- Policies that can block a stage change. An evidence log of who acted, person or agent.
- Acceptance fields for a threshold, a dataset label, and a latency target.

Agents, data, and shop setup

- A timeboxed research item with one question: prompt, deeper agent, or data. The result is proceed, reshape, or drop.
- A feature that needs data cannot enter Build until that data check proceeds, or a product owner records an exception.
- An agent feature records a risk tier, the tools it may call, a held-out eval reference, who can stop it, and a rollback drill.
- A prompt or model change sends the feature back to Check. An incident marks the agent advisory and can add an eval case.
- SQLite on a laptop. A shop can switch to PostgreSQL, SQL Server, Oracle, or another SQL database through an adapter.
- Local login and JWT. Roles stay in this database. A shop can replace the login with its own code.
- A plugin for the shop's own tool. Cursor, Claude, or another product can submit notes and documents as proposed initiatives. The product owner accepts them for the scrum master and the team.
- A story pack exported for that same tool, so developers can ask it to write code. The code stays in the shop's repository.
- A short in-app tutorial and a guide a product manager can read away from the screen.

Process steps

The steps below are the whole loop. A small shop does not get a shorter loop. It leaves optional seats empty and still walks the steps that remain.

Set up once

1 **Name the stream and the teams**
Create a portfolio, a value stream, and at least one team. Create a delivery group, or accept the placeholder that stands in for one.

2 **Assign roles**
Give each person the roles they will act in. A product manager may also be product owner and scrum master. Add group facilitator only if someone will manage the group.

Open a planning cycle

3 **Align on outcomes**

In one short session, record group outcomes, team outcomes, milestones, risks, and a confidence check. Give every dependency an owner and a needed-by date. Name a providing team only if you want to.

4 **Write dependent work when it applies**

If a feature needs a firewall change, database access, or any other team, write what must be done, what done looks like, and the needed-by date. Use a team name you already created, or leave the team blank. One team may cover more than one kind of access. Skip this step when no other team is involved. Mark an architecture review or a PII review only on the features that need them.

5 **Plan inside each team**

Each team opens an iteration inside the cycle, estimates its stories in t-shirt sizes or points, and orders its own backlog. The shared session does not plan every task.

Move one piece of work

6 **Discover**

Write the problem and the outcome the work is for.

7 **Research, when the work needs it**

If the feature needs data, or may use an agent, open a research item. Set a clock, five days unless you choose otherwise. At the end, record proceed, reshape, or drop. A deeper-agent question waits until a prompt question has failed.

8 **Define**

Write the bounded intent. A person approves it. That approval is the limit of what the team or an agent may do.

9 **Build**

Build starts only after Define is approved. If the feature needs data, Build also waits for a data decision of proceed, or for a recorded exception.

10 **Check**

Someone other than the producer completes Check. The shared done list must be clear: integrated, acceptance met, and no dependency left without an owner. An architecture review or a PII review blocks Check only when the feature said that review applies and it is still open.

11 **Ship**

A named person decides to ship. A passing check is evidence for that decision. It is not the decision. Prototypes and experiments stop at Check unless someone explicitly ships them.

During the iteration

12 **Act as product owner**

Order the backlog and accept a story only while acting as product owner.

13 **Act as scrum master**

Write the team sync, an impediment, or the retrospective only while acting as scrum master. The retro stays on the team unless the team promotes one learning.

14 **Name cross-team issues**
In the cross-team note, record the impediment or dependency and which team is asked to respond. Naming the team is optional.

15 **Respect the work-in-progress limit**
Do not start another item while the board is at its limit. The scrum master can raise the limit for one item by recording a reason.

At the iteration boundary

16 **Review the evidence**
Record what was observed: a count, a customer comment, a demo note. Then record a decision: continue, drop, reshape, or resize. The count can include the iteration's velocity.

17 **Leave private notes private**
Raw retrospective text is not copied into the evidence review. Only a learning the team promotes can be attached.

After an agent ships

18 **Re-check when the agent changes**
A new prompt version or model version returns the feature to Check.

19 **Handle an incident**
Mark the agent advisory until a person closes the incident. Add the failed case to the eval reference.

What is required

These stay in place for a two-team shop and for a larger one. Skipping them would be a different process.

Always

- A value stream and at least one team.
- A delivery-group record, even when it is only a placeholder. Cycles and dependencies need a parent.
- A product owner for backlog order and story acceptance.
- A scrum master for the team sync, impediments, and the retrospective.
- An acting role on every saved task. The person switches to the role the task belongs to.
- For work that will ship: Discover, an approved Define, Build, an independent Check, and a human Ship.
- The shared done list, clear before Check can finish.
- An owner and a needed-by date on every dependency that is written down. The providing team may be blank.
- A sizing method on each team, t-shirt or points, and an estimate on each story that team plans.
- A work-in-progress limit on the team board.
- An evidence record of who acted, and whether they were a person or an agent.

When the feature needs data

- A data research item with a timebox and a decision. Build stays closed on drop. Proceed requires a sample, access, and a named owner.
- An exception, with a reason, if a product owner moves the feature forward without that decision. The exception stays visible.

When the feature ships an agent

- A risk tier: assistive, read-only, changes data, or high impact.
- The list of tools the agent may call, and which actions still need a person.
- A held-out eval reference. These are cases the prompt was not written against. A failed critical case blocks Ship.
- A named person who can stop the agent, and a note that rollback of the prompt, the tool list, and the policy was drilled.
- A return to Check when the prompt version or the model version changes.

What can be skipped

Skipping here means the same process with an empty seat or a step that does not apply. It does not mean a second, lighter method.

Optional inside the process

- Dependent work. Leave it off when no other team has to act. When you write it, the team name is still optional.
- An architecture review, and a PII compliance review. Both stay off unless the feature needs that review.
- A dedicated group facilitator. Leave the seat empty. Tasks that manage the group turn off. Team steps stay.
- A fully staffed delivery group. Use the placeholder. Turn it into an active group later without moving the teams or the cycles.
- Shipping a prototype or an experiment. They end at Check unless someone chooses to ship.
- A deeper-agent research item, until a prompt trial has failed its question.
- A second research question on work that is ordinary delivery and does not need data or an agent.
- Cloud login. Local login is enough to run the tool. JWT is available when a shop wants it.
- A shop database on the first day. SQLite is the local default. PostgreSQL, SQL Server, Oracle, or another SQL adapter can wait.

Not in this tool

These are real practices. They belong in other systems. Scaled Engine stores the decision and the gate, not the runtime.

- Running a model, storing prompt text, scoring an eval, or hosting the agent's tools.
- Token-cost accounting, penetration tests, and a security program.
- Azure AD and AWS IAM as built-in logins. A shop may replace the authenticator with its own code. Roles still live in this database.

- Calling Cursor, Claude, or any other model. The shop's tool does that, then pushes a proposal in. Their source repository is never opened.
- In-product helpers that draft dependencies, splits, scores, or risk states. A shop that already has a tool uses the plugin instead.
- Funding guardrails and an investment workshop.
- A separate fast engine, and a container image, until a measured path needs them.
- A course, quizzes, or certificates. Training is the in-app tutorial plus this overview.

> **The rule for a small shop**
>
> Same person, same steps. A product manager may be product owner and scrum master.
>
> Backlog work is still done as product owner. The retro is still done as scrum master.
>
> Managing the group is still done as facilitator, or that seat stays empty.